

Scenario

CartStream Analytics powers real-time monitoring and dynamic merchandising for an international e-commerce marketplace. The data platform currently ingests raw user interactions (page views, cart additions, and purchases) as a continuous event stream. As a Junior Data Platform Engineer, your task is to build a real-time stream processing pipeline using Apache Flink SQL: define streaming tables with event-time watermarking, and analyze real-time sales using tumbling window table-valued functions (Windowing TVFs) to deliver aggregate metrics to internal analytics dashboards.

Graduated Sub-Tasks

Task 1 (Easy / Foundational) — Streaming Source DDL, Event Time & Watermarking

1. Define a Flink SQL source table named `ecommerce_events` that reads continuous e-commerce activity records formatted in CSV/JSON.
2. Structure the schema to map the primary interaction fields:
 - `event_time_str`: Original string timestamp (VARCHAR).
 - `event_time`: Computed timestamp converted to native `TIMESTAMP(3)` using `TO_TIMESTAMP(event_time_str, 'yyyy-MM-dd HH:mm:ss z')`.
 - `event_type`: Type of event (view, cart, remove_from_cart, purchase).
 - `product_id`: Identifier for the inventory item (BIGINT).
 - `category_code`: Taxonomy classification string (VARCHAR).
 - `brand`: Brand identifier (VARCHAR).
 - `price`: Unit price of the product (DECIMAL(10, 2)).
 - `user_id`: Customer identifier (BIGINT).
3. Configure **Event Time** processing by declaring a watermark strategy on `event_time`:
4. `$$\text{WATERMARK FOR } event\_time \text{ AS } event\_time - \text{INTERVAL '5' SECOND}$$$`
5. Explain how this 5-second watermark boundary ensures deterministic window processing for out-of-order event arrivals.

Task 2 (Medium / Applied) — Windowing TVFs & Real-Time Revenue Aggregation

1. Using Flink's modern Windowing Table-Valued Function (`TABLE(TUMBLE(...))`), implement a non-overlapping 5-minute tumbling window aggregation on `ecommerce_events`.
2. Filter the incoming stream solely for completed purchase transactions (`event_type = 'purchase'`).
3. Aggregate the streaming window to compute the following metrics grouped by `window_start`, `window_end`, and `brand`:
 - `total_orders`: Count of completed purchase events within the window.
 - `gross_revenue`: Sum of `price` values within the window, rounded to 2 decimal places.
 - `avg_order_value`: Average transaction value within the window.
 - `unique_buyers`: Approximate distinct count of active purchasers (`COUNT(DISTINCT user_id)`).
4. Direct the streaming output to an append-only sink table (`brand_window_sales`) using Flink's `print` or `filesystem` connector.

Requested Deliverables

- Executable Flink SQL DDL and Query Script (`flink_ecommerce_pipeline.sql`) or complete PyFlink streaming script (`flink_ecommerce_streaming.py`).
- Visual execution verification log displaying running window outputs from the Flink SQL client or PyFlink console runner.

Mandatory Dataset Source

- **Direct Kaggle Access Link:** [eCommerce Events History in Cosmetics Shop on Kaggle](#)
- **Dataset Characteristics:**
 - **Rows:** Over 4 million real-world customer activity events.
 - **Primary Schema Fields:** `event_time`, `event_type`, `product_id`, `category_id`, `category_code`, `brand`, `price`, `user_id`, `user_session`.
 - **Intentional Flaws:** Missing values in `brand` and `category_code` (null strings), out-of-order event timestamps requiring watermark tolerance, and bursty purchase activity.